

Deep Learning Boston Housing Prediction — Full Theory

Aim

The aim of this project is to predict Boston house prices using a Deep Learning Neural Network built with TensorFlow and Keras.

The model learns relationships between house features such as:

- number of rooms
- crime rate
- tax rate
- area
- population

and predicts the house price.

Overall Flow of Program

Import Libraries

↓

Load Dataset

↓

Preprocess Data

↓

Split Dataset

↓

Create Neural Network

↓

Train Model

↓

Evaluate Model

↓

Predict Output

↓

Display Graphs

1. Importing Libraries

Libraries used:

- TensorFlow
- NumPy
- Pandas
- Matplotlib

Purpose

TensorFlow

Used for building and training neural networks.

NumPy

Used for numerical calculations and arrays.

Pandas

Used for handling dataset tables.

Matplotlib

Used for plotting graphs and visualization.

2. Loading Dataset

The Boston Housing dataset is loaded.

The dataset contains:

- input features (X)
- target house prices (y)

Each row represents one house.

The dataset contains 13 input features.

3. Data Preprocessing

Before training, data is normalized using scaling.

Example:

- room count may be 5
- tax may be 500
- population may be 10000

Large differences in values can confuse the model.

So scaling converts all features into similar ranges.

Advantages

- faster training
 - stable learning
 - better accuracy
-

4. Splitting Dataset

Dataset is divided into:

- training data
 - validation data
 - testing data
-

Training Data

Used to teach the model.

Validation Data

Used during training to check whether learning is improving.

Testing Data

Used only after training for final evaluation.

It acts like a final exam.

5. Neural Network Architecture

The model is created using Sequential model in Keras.

It contains:

- input layer
 - hidden layers
 - output layer
-

Input Layer

The input layer takes 13 features from dataset.

`Input(shape=(13,))`

Hidden Layers

Dense hidden layers are used:

`Dense(128)`

`Dense(64)`

`Dense(32)`

These layers learn patterns and relationships in data.

More neurons increase learning capability.

Activation Function

The hidden layers use:

ReLU Activation Function

Full form:

Rectified Linear Unit

Formula:

$$f(x) = \max(0, x)$$

Working of ReLU

- positive value $\rightarrow$ remains same
- negative value $\rightarrow$ converted to 0

Example:

Input = 5 $\rightarrow$ Output = 5

Input = -3 $\rightarrow$ Output = 0

Why ReLU is Used

Advantages:

- simple and fast
 - computationally efficient
 - helps learn complex patterns
 - avoids vanishing gradient problem
 - improves training speed
-

Output Layer

The output layer predicts house price.

Since this is a regression problem:

- only one output neuron is needed

Usually linear activation or no activation is used.

6. Dropout Layer

Dropout is used to prevent overfitting.

Example:

Dropout(0.3)

During training:

- some neurons are randomly disabled

This forces the network to learn generalized patterns instead of memorizing data.

7. Batch Normalization

Batch Normalization stabilizes and speeds up training.

Advantages:

- faster convergence
 - smoother learning
 - improved performance
-

8. Model Training

Training is done using:

`model.fit()`

During training:

1. model takes input
 2. predicts output
 3. compares prediction with actual answer
 4. calculates error
 5. updates internal weights
 6. repeats process many times
-

Epoch

An epoch means:

One complete pass through the entire training dataset.

Example:

If dataset has 1000 rows:

epochs = 10

means:

the model studies all 1000 rows 10 times.

Batch Size

Instead of taking full dataset together, training data is divided into smaller batches.

Example:

`batch_size = 100`

Dataset:

1000 rows

Training happens as:

Batch 1 → 100 rows
Batch 2 → 100 rows
...
Batch 10 → 100 rows

Why Batches are Used

Advantages:

- less memory usage
 - faster computation
 - smoother learning
 - efficient GPU usage
-

Iterations

One batch processed once is called one iteration.

Formula:

$$\text{Iterations per Epoch} = \frac{\text{Total Samples}}{\text{Batch Size}}$$

Example

If:

- total samples = 1000
- batch size = 100

then:

$$\frac{1000}{100} = 10$$

So:

- 10 iterations complete 1 epoch.
-

Loss Function

Loss measures prediction error.

Lower loss means better predictions.

The model uses:

Mean Squared Error (MSE)

Formula:

$$MSE = \frac{1}{n} \sum (y_{\text{actual}} - y_{\text{predicted}})^2$$

Working of MSE

1. Find difference between actual and predicted values
2. Square the error
3. Find average of all squared errors

Smaller MSE means better model.

Backpropagation

Backpropagation is the learning mechanism of neural network.

Steps:

1. prediction is made
2. error is calculated
3. error moves backward through network
4. weights are adjusted to reduce future error

This process improves predictions gradually.

Optimizer

The optimizer used is:

Adam Optimizer

Purpose:

- updates weights efficiently
- reduces loss
- improves learning speed

Advantages:

- fast
 - stable
 - widely used in deep learning
-

Learning Rate

Learning rate controls:

how big the update steps are during learning.

- small learning rate → slow learning
 - very large learning rate → unstable training
-

Overfitting

Overfitting happens when:

- model memorizes training data
- performs poorly on new data

Symptoms:

- high training accuracy
 - low testing accuracy
-

Preventing Overfitting

1. Dropout

Randomly disables neurons during training.

2. EarlyStopping

Stops training automatically when validation performance stops improving.

This prevents unnecessary training and memorization.

Evaluation Metrics

After training, model performance is checked using:

- MAE
 - RMSE
 - R^2 Score
-

MAE

Mean Absolute Error.

Measures average prediction error.

Lower MAE means better predictions.

RMSE

Root Mean Square Error.

Measures prediction accuracy.

Smaller RMSE means predicted values are closer to actual values.

R² Score

Measures how well model fits the data.

Value closer to 1 means better performance.

Prediction

After training:

```
model.predict()
```

is used to predict house prices for unseen data.

Graphs

Matplotlib graphs are used to show:

- training loss
- validation loss
- model improvement over epochs

If loss decreases gradually:

- model is learning properly.
-

Final Conclusion

This project uses Deep Learning Neural Networks to predict Boston house prices. Data preprocessing, normalization, dense hidden layers, ReLU activation, dropout, backpropagation, Adam optimizer, and MSE loss function are used to train the model effectively. The trained model predicts house prices with good accuracy and performance.

